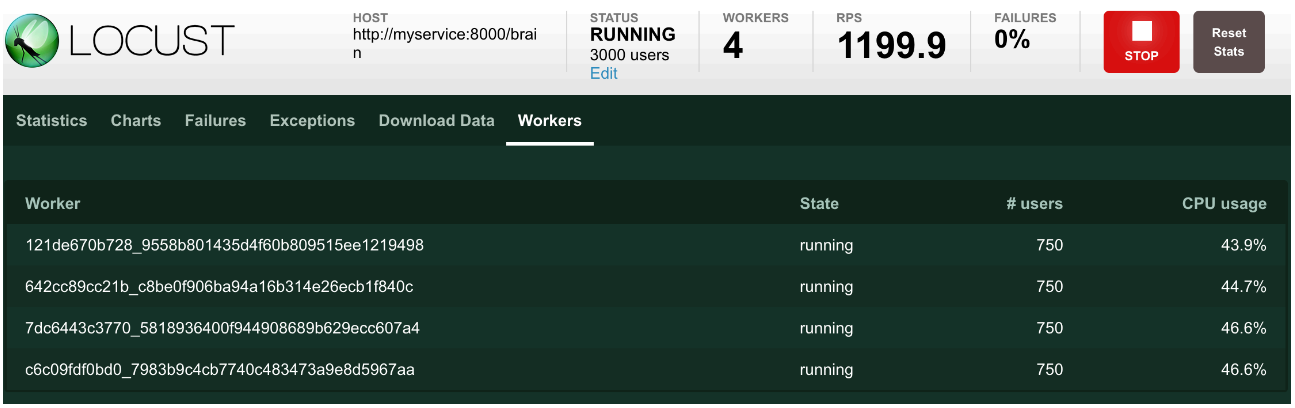
How to increase RPS in distributed locust load test

Asked 5 years, 7 months ago Modified 3 years, 8 months ago Viewed 12k times

5

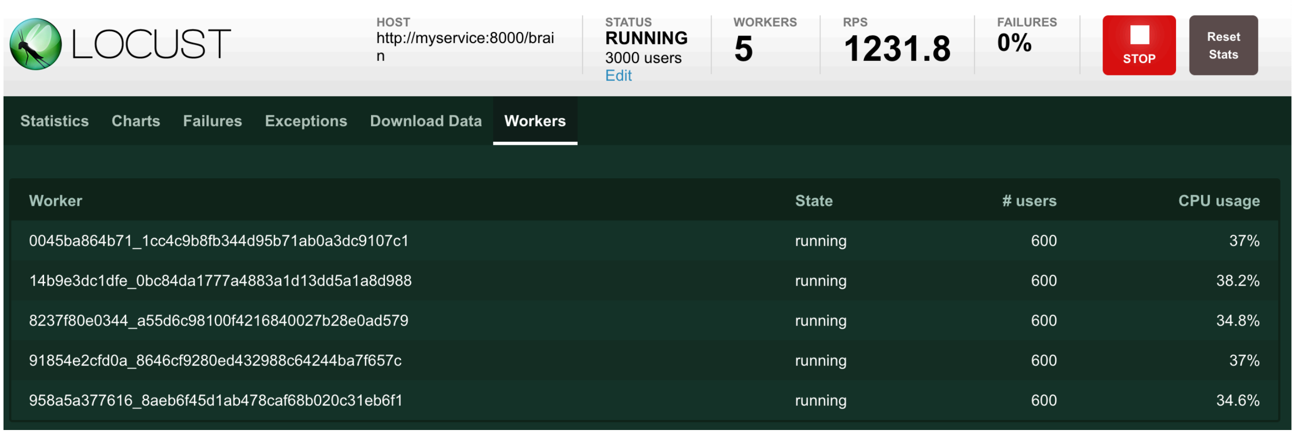
I cannot get past 1200 RPS no matter if I use 4 or 5 workers.

I tried to start locust in 3 variations -- one, four, and five worker processes (`docker-compose up --scale worker_locust=num_of_workers`). I use 3000 clients with a hatch rate of 100. The service that I am loading is a dummy that just always returns `yo` and HTTP 200, i.e., it's not doing anything, but returning a constant string. When I have one worker I get up to 600 RPS (and start to see some HTTP errors), when I have 4 workers I can get up to the ~1200 RPS (without a single HTTP error):



Worker	State	# users	CPU usage
121de670b728_9558b801435d4f60b809515ee1219498	running	750	43.9%
642cc89cc21b_c8be0f906ba94a16b314e26ecb1f840c	running	750	44.7%
7dc6443c3770_5818936400f944908689b629ecc607a4	running	750	46.6%
c6c09fdf0bd0_7983b9c4cb7740c483473a9e8d5967aa	running	750	46.6%

When I have 5 workers I get the same ~1200 RPS, but with a lower CPU usage:



Worker	State	# users	CPU usage
0045ba864b71_1cc4c9b8fb344d95b71ab0a3dc9107c1	running	600	37%
14b9e3dc1dfe_0bc84da1777a4883a1d13dd5a1a8d988	running	600	38.2%
8237f80e0344_a55d6c98100f4216840027b28e0ad579	running	600	34.8%
91854e2cfd0a_8646cf9280ed432988c64244ba7f657c	running	600	37%
958a5a377616_8aeb6f45d1ab478caf68b020c31eb6f1	running	600	34.6%

I suppose that if the CPU went down in the 5-worker case (with respect to 4-worker case), than it's not the CPU that is bounding the RPS.

I am running this on a 6-core MacBook.

The `locustfile.py` I use posts essentially almost empty requests (just a few parameters):

Copy

```python
from locust import HttpUser, task, between, constant


class QuickstartUser(HttpUser):
    wait_time = constant(1)  # seconds

    @task
    def add_empty_model(self):
        self.client.post(
            "/models",
            json={
                "grouping": {
                    "grouping": "a/b"
                },
                "container_image": "myrepo.com",
                "container_tag": "0.3.0",
                "prediction_type": "prediction_type",
                "model_state_base64": "bXkgc3RhdGU=",
                "model_config": {},
                "meta": {}
            }
        )
```

My docker-compose.yml:

Copy

```yaml
services:
  myservice:
      build:
          context: ../
      ports:
          - "8000:8000"

  master_locust:
      image: locustio/locust
      ports:
          - "8089:8089"
      volumes:
          - ./:/mnt/locust
      command: -f /mnt/locust/locustfile.py --master

  worker_locust:
      image: locustio/locust
      volumes:
          - ./:/mnt/locust
      command: -f /mnt/locust/locustfile.py --worker --master-host master_locust
```

Can someone suggest the direction of getting towards the 2000 RPS?

load-testing locust

Share Improve this question Follow edited Oct 30, 2020 at 18:10 asked Oct 30, 2020 at 10:13

Anton Daneyko
6,552 8 37 66

Hi. I have same problem. I've tried one, four and five workers using respectively one, two and three computers, but the RPS is still the same (about 200). Did you find the solution? – Hamidreza Apr 11, 2021 at 11:58

There's some unresolved misunderstanding between how I understand things work and the way the only answer suggest. My understanding of such test is the following: if my service can handle one request per second and if this server is bombarded by 1000 request each second by the locust workers, then I expect to see 999 error responses and 1 success, given the workers have one second timeout. Based on the comment below, it seems that my understanding is not what the locust actually does, but I don't get what @Cyberwiz said. – Anton Daneyko Apr 16, 2021 at 11:25

2 Answers

Sorted by: Highest score (default)

2

You will need more workers to generate more RPS. I thought one worker will have limited local port range when creating tcp connection to the destination.

You may check this value in your linux worker:

```
net.ipv4.ip_local_port_range
```

Try to tweak that number it on your each linux worker, or simply create hundreds of new worker with another powerful machine (your 6-core cpu macbook is to small)

To create many workers you could try Locust in kubernetes with horizontal pod autoscaling for the workers deployment.

Here is some helm chart to start play arround with Locust k8s deployment: https://github.com/deliveryhero/helm-charts/tree/master/stable/locust

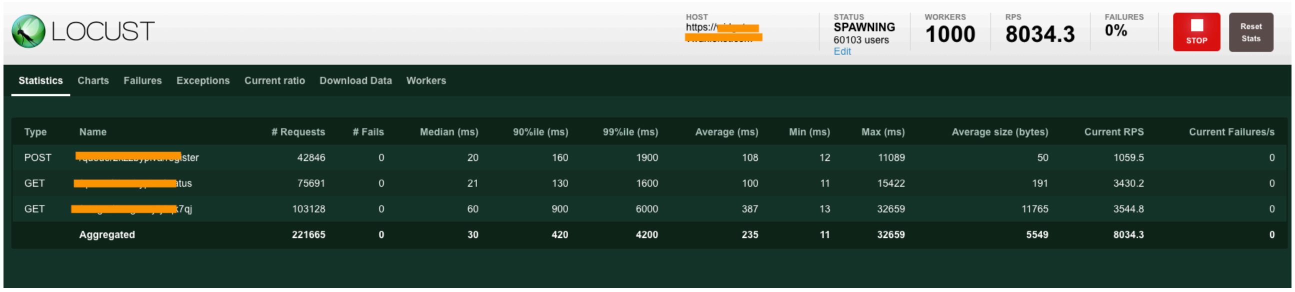
You may need to check these args for it:

```
worker.hpa.enabled
worker.hpa.maxReplicas
worker.hpa.minReplicas
worker.hpa.targetCPUUtilizationPercentage
```

simply set the `maxReplicas` value to get more workers when the load testing is started. Or you can scale it manually with `kubectl` command to scale worker pods to your desired number.

I've done to generate minimal 8K rps (stable value for my app, it can't serve better) with 1000 pods/worker, with Locust load test parameter like 200K users with 2000 spawn per second.

You may have to scale out your server when you reach higher throughput, but with 1000 pods/worker i thought you can easily reach 15K-20K rps.



Type	Name	# Requests	# Fails	Median (ms)	90%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[redacted]ster	42846	0	20	160	1900	108	12	11089	50	1059.5	0
GET	[redacted]tus	75691	0	21	130	1600	100	11	15422	191	3430.2	0
GET	[redacted]7qj	103128	0	60	900	6000	387	13	32659	11765	3544.8	0
	Aggregated	**221665**	**0**	**30**	**420**	**4200**	**235**	**11**	**32659**	**5549**	**8034.3**	**0**

Share Improve this answer Follow

answered Oct 3, 2022 at 22:06

racbear

31 2

Sign up to request clarification or add additional context in comments.

Comments

Add a comment

1

You should check out the FAQ.

https://github.com/locustio/locust/wiki/FAQ#increase-my-request-raterps

It's probably your server not being able to handle more requests, at least from your one machine. There are other things you can do to make more sure that's the case. You can try FastHttpUser, running on multiple machines, or just upping the number of users. But if you can, check to see how the server is handling the load and see what you can optimize there.

Share Improve this answer Follow

answered Oct 30, 2020 at 14:49

Solowalker

2,936 1 11 14

3 Comments

Add a comment

Anton Daneyko Over a year ago

AFAI the RPS is generated by workers and has nothing to do with the server that serves the requests. If my server can handle 1 request per second and I flood it with 1000 RPS with 2 workers then I am expecting to see 1 served request and 999 errors. My point is that the capability of the server is out of the scope here.

2 Reply

Cyberwiz Over a year ago

Sorry, but that assumption is most often incorrect, as servers almost always implement a queue and/or try to execute more concurrent requests than they have CPU cores for. If response times increase as your load goes up then you need to investigate on the server side first, as that is your most likely culprit. Only if your server says your response times are good (check the access log or whatever), but locust says they are bad then MAYBE you can blame locust :)

2 Reply ...

Anton Daneyko Over a year ago

@Cyberwiz Thanks for your input Lars! Is RPS -- a request per second or response per second?

0 Reply ...